

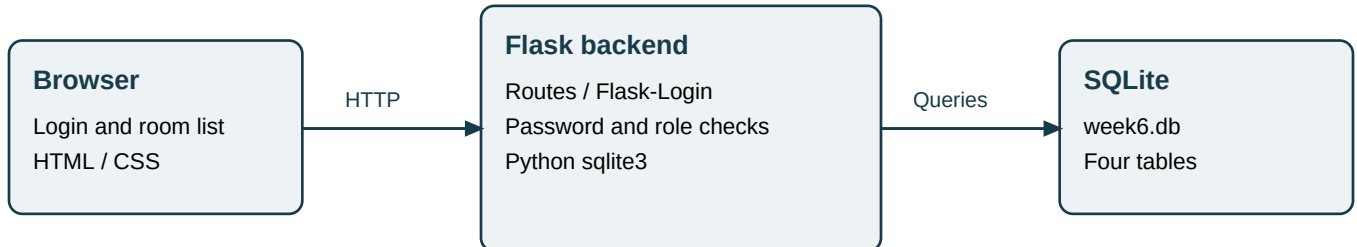
Secure Design

BIBIFI Group 11 | Huseyin Simsek and Gustavo Pepe

Weeks 4-6 - Secure Art Gallery - Fall 2026

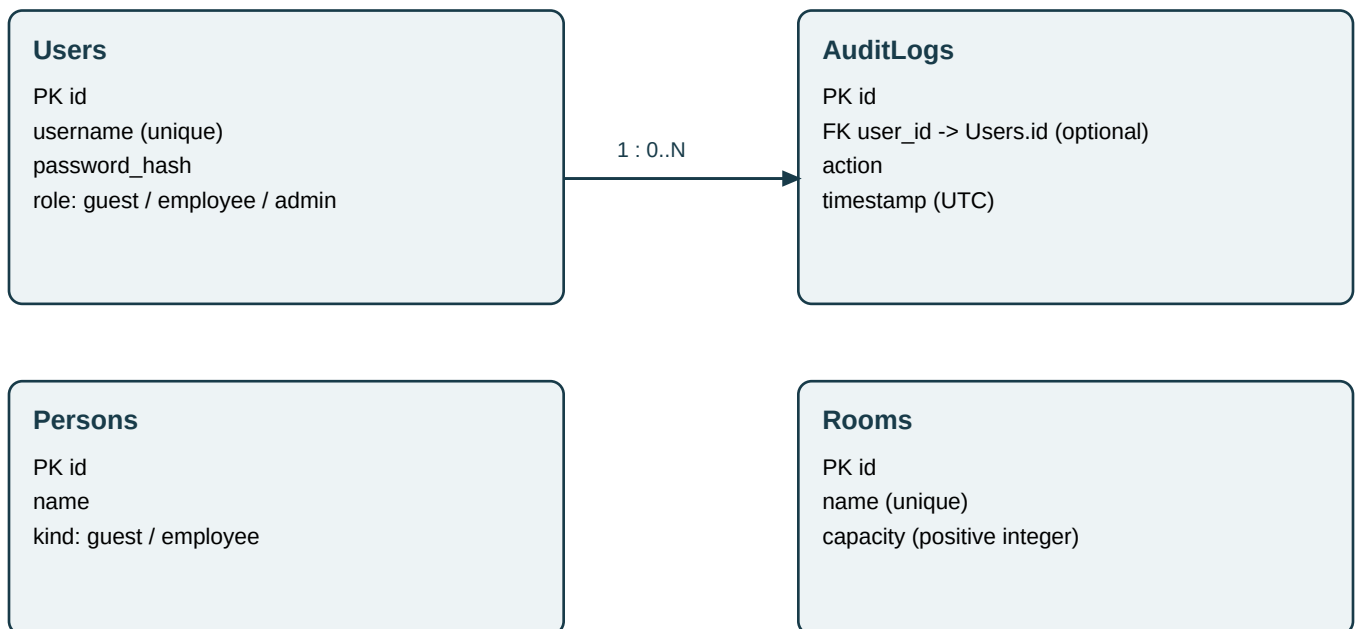
Architecture

One Flask application renders Jinja pages, verifies login and roles, and queries SQLite through Python's built-in sqlite3 module. A separate REST service is unnecessary for this milestone.



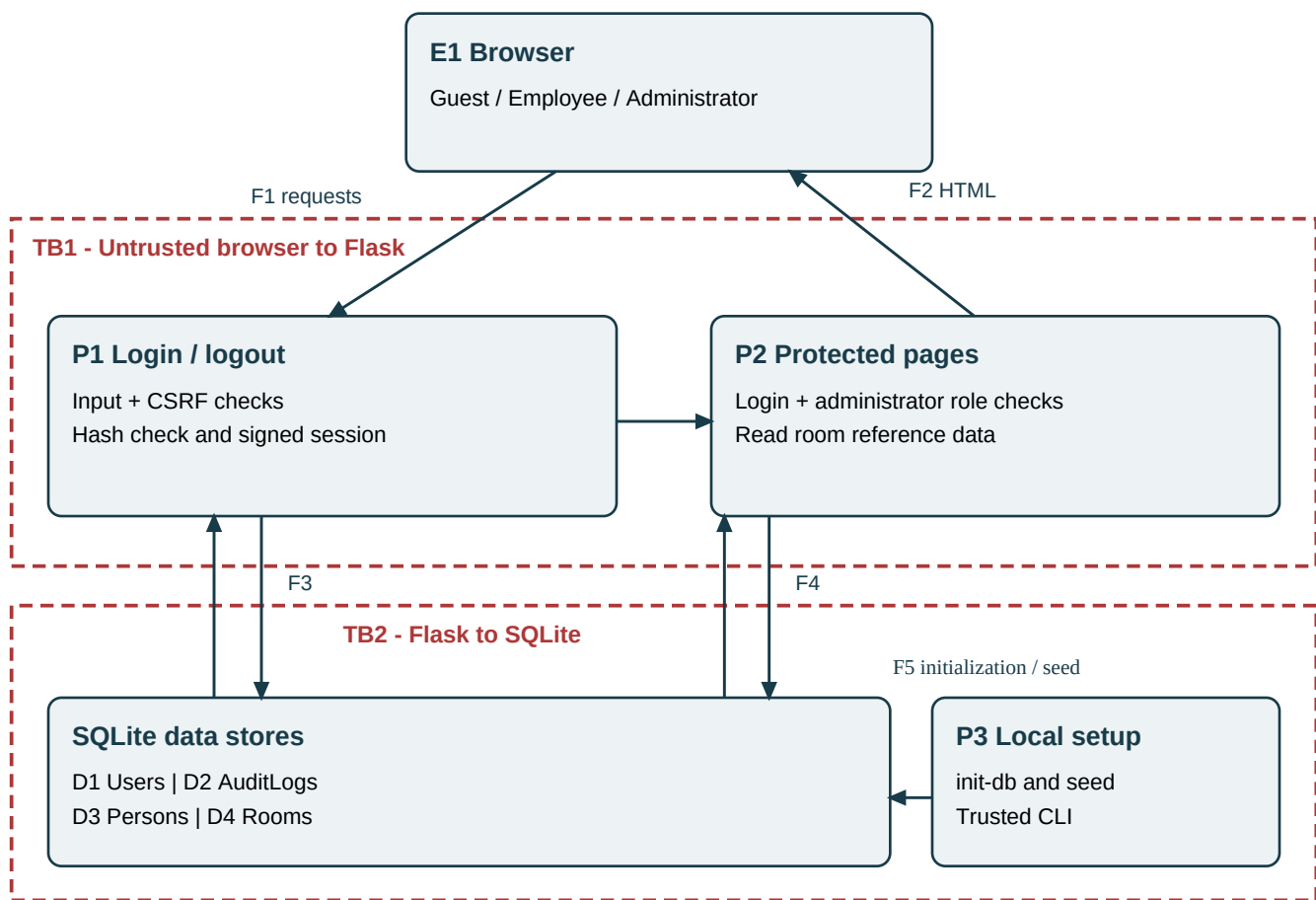
Browser requests return server-rendered HTML. Local demonstration: 127.0.0.1.

Entity relationship diagram



Users store login accounts. Persons and Rooms are seeded gallery reference records. Users 1 -> 0..N AuditLogs; an AuditLog may have no user when the submitted username is unknown. Persons and Rooms have no relationship to login accounts in this milestone. No GalleryEvents or session table is needed: Flask supplies signed sessions.

Data-flow diagram



Flows and trust boundaries

Flow	Data and controls
F1 / F2	Browser sends login/logout requests and its session cookie; Flask returns HTML. CSRF and server validation check form submissions.
F3	Authentication reads Users, verifies a password hash, and appends an authentication log.
F4	Protected routes load the account role and read Rooms. The administrator route checks the role on the server.
F5	The local setup commands create tables and seed hashed accounts plus fictional reference records.

TB1 - Browser to Flask: browser input, cookie contents and URLs are untrusted. Signed sessions, login checks, CSRF and server-side authorization enforce the boundary. Hiding a link is not authorization.

TB2 - Flask to SQLite: only the application and local setup commands access the database file. sqlite3 binds browser values using ? placeholders. Credentials and database files are outside static assets and excluded from Git.

Local HTTP uses 127.0.0.1. Encryption in transit is not provided by this milestone. The local computer and its file permissions are part of the trusted development environment.

STRIDE threat model

Each threat identifies a component, a concrete scenario, its impact and the mitigation. These are design observations; limited controls do not establish resistance to every attack.

Threat / component	Attack scenario and impact	Mitigation and status
Spoofing - login and Users	An attacker guesses credentials or obtains the database and tries to impersonate an account. Impact: unauthorized login.	Implemented: salted script hashes, credential checks and generic login errors. Login rate limiting is not included.
Tampering - login/logout forms	A malicious site submits a forged form, or a client changes submitted values. Impact: unintended session changes or invalid input.	Implemented: CSRF tokens, input validation and parameterized SQLite queries. Only login/logout forms remain.
Repudiation - authentication log	A user denies logging in or out. Impact: no basic record for troubleshooting.	Implemented: timestamped LOGIN_SUCCESS, LOGIN_FAILED and LOGOUT records. Local database owners can alter these records; no tamper-proof logging is claimed.
Information Disclosure - templates and repository	An attacker inspects responses or committed files for passwords and secrets. Impact: credential or account exposure.	Implemented: password hashes, generic failed-login messages, Jinja escaping, and ignored secret/database files. Local HTTP has no transport encryption.
Denial of Service - Flask login	An attacker sends oversized requests or repeats expensive password checks. Impact: resource exhaustion or slow login.	Implemented: 16 KiB body limit. Repeated small requests remain a limitation; rate limiting and broader defenses are deferred.
Elevation of Privilege - administrator route	An Employee opens /admin directly or tampers with a session cookie. Impact: unauthorized administrator access.	Implemented: signed sessions, database-loaded user role, login_required and an explicit server-side admin check.

Security requirements and evidence

SEC-1 through SEC-9 are defined in phase1_requirements.pdf. database/schema.sql defines the tables and constraints. db.py opens and closes SQLite connections and records authentication activity. routes.py verifies passwords and roles. __init__.py configures sessions/CSRF and initializes and seeds the database. Templates escape output and include CSRF fields.

The functional and access tests verify only the Week 6 milestone. No event processing, account-management interface, production deployment or full security assessment is included.

Source and scope decision

F26_BIBIFI_Project.pdf, pages 3-5, defines the design and Week 6 checklist. Page 5 assigns gallery events, status, persons management, history, administrative functions and state validation to Week 7. Page 7 lists the overall repository layout; Docker support is omitted from this deliberately limited Week 4-6 version.